

Soluții aplicate — auth prin Kong + Swagger

Repo [nimigeanconstantinon/ms-gitops](#) · 2026-07-22 · ce a fost aplicat + runbook de bootstrap

Ce s-a schimbat în repo

#	Fișier	Acțiune
C1	<code>bootstrap/root.yaml:14</code>	Scos comentariul leftover
M1	<code>realm-rsk.yaml</code>	Scos clientul <code>spring-api</code> (secret în clar, neutilizat)
M3	<code>data-service/values.yaml:44</code>	<code>ingress.enabled: true → false</code>
M2	<code>realm-rsk.yaml</code>	+ client <code>oauth2-proxy</code> + mapper <code>groups</code> + grup <code>/admins</code> + <code>admin</code> în grup
M2	<code>business/rsk/oauth2-proxy/</code>	Nou: deployment + service + ingress + README
M2	<code>argo-apps/app-oauth2-proxy.yaml</code>	Nou: Application (wave 4)

M2 — de ce fără Crossplane

car-platform exportă `client-secret`-ul prin Crossplane. Constantin **nu are Crossplane**, deci adaptat la sealed-secrets:

REALM — CLIENT FĂRĂ SECRET ÎN FIȘIER

```
- clientId: oauth2-proxy
  publicClient: false
  standardFlowEnabled: true
  redirectUris:
    - "https://data-service.icode.mywire.org/oauth2/callback"
  # FARA camp secret -> Keycloak il genereaza, il sigilezi tu
  protocolMappers:
    - name: groups
      protocolMapper: oidc-group-membership-mapper
      config: { "claim.name": "groups", "full.path": "true", ... }
```

Fără clear-text: secretul nu apare în git; stă într-un `SealedSecret` pe care îl produci tu (nu poate fi sigilat de review — e legat de cheia cluster-ului).

M3 — before / after

```
# business/rsk/data-service/values.yaml
ingress:                                # BEFORE — a doua usa, fara auth
  enabled: true
  host: data.icode.mywire.org
ingress:                                # AFTER — singura usa e Kong (gated de oauth2-proxy)
  enabled: false
```

Runbook bootstrap (o singură dată)

1 · RE-IMPORTĂ REALMUL (KEYCLOAKREALMIMPORT E CREATE-ONLY!)

```
kubectl -n auth delete keycloakrealmimport rsk-realm # ArgoCD îl recreează din git
```

2 · IA CLIENT-SECRET-UL GENERAT DE KEYCLOAK

```
CID=$(kcadm.sh get clients -r rsk -q clientId=oauth2-proxy --fields id --format csv --noquotes | tail -1)
kcadm.sh get clients/$CID/client-secret -r rsk --fields value
# sau: admin console -> realm rsk -> Clients -> oauth2-proxy -> Credentials
```

3 · GENEREAZĂ COOKIE-SECRET

```
openssl rand -base64 32
```

4 · SIGILEAZĂ ȘI COMITE

```
kubectl create secret generic oauth2-proxy-keycloak \
  --namespace business \
  --from-literal=client-secret='<PAS_2>' \
  --from-literal=cookie-secret='<PAS_3>' \
  --dry-run=client -o yaml \
| kubeseal --controller-namespace sealed-secrets --format yaml \
> business/rsk/oauth2-proxy/sealed-secret.yaml
```

Verificare

```
kubectl -n business get deploy,pod -l app=oauth2-proxy
curl -sI https://data-service.icode.mywire.org/swagger-ui/ # 302 -> Keycloak login (NU 200)
```

Login cu `admin/admin` (în `/admins`) → trece. `test` (fără grup) → respins. Asta confirmă poarta 1 + mapper-ul `groups`.

Toate fișierele validate YAML. Git neatins — commit/push le faci tu. Ordine: push cod → re-import realm → seal secret → verificare.